GPUMODE Lecture Study Notes: Lecture 16 On Hands-On Profiling

Core Problem the Lecture Addresses

This lecture is about how to profile and optimize a real PyTorch training workload by moving between several levels of abstraction:

- high-level model code in Python,
- PyTorch runtime operations,
- CUDA kernel launches,
- GPU stream timelines,
- host-device synchronization points,
- memory allocation timelines,
- and hardware roofline-style estimates.

The central problem is not merely "how do I make a kernel faster?" The deeper problem is:

> A model can appear to be using the GPU, and individual CUDA kernels can appear reasonable, while the overall system is still underutilizing the hardware because of host overhead, synchronization, poor batching, memory-bound matrix multiplications, variable sequence lengths, and Python object lifetime issues.

The lecture uses a fine-tuning workload for the Gemma model inside Lightning AI Studios. The session is intentionally hands-on: instead of beginning from slides, Taylor Robie profiles an actual model, inspects the traces, forms hypotheses, validates them with microbenchmarks and profiler evidence, and then identifies practical optimizations.

The key lesson is that performance work requires moving between views:

- NVIDIA SMI can tell us whether the GPU is doing something.
- Nsight Systems can show CPU and GPU timelines, CUDA streams, kernel gaps, and occupancy.

- The PyTorch profiler can connect kernels back to PyTorch operations and source code.
- PyTorch memory snapshots can reveal allocation lifetimes and anomalous memory behavior.
- Datasheet calculations can tell whether a kernel is compute-bound or memory-bandwidth-bound.

The lecture repeatedly emphasizes that optimization should be evidence-driven. A kernel being visible in a trace does not automatically mean it is the right thing to optimize. A GEMM dominating time does not automatically mean the model is compute-bound. A fused kernel being faster on-device does not automatically mean the end-to-end system becomes faster.

Required Background

GPU Execution Model

A CUDA program launches kernels onto a GPU. Each kernel is executed by a grid of thread blocks, and each thread block is scheduled onto a streaming multiprocessor, or SM. Threads inside a block are grouped into warps, usually of size 32 threads on NVIDIA GPUs.

Important execution terms are:

- **Thread**: the smallest unit of CUDA execution.
- **Warp**: a group of usually 32 threads executing in SIMT fashion.
- **Thread block**: a group of threads that can cooperate through shared memory and synchronization.
- **Grid**: the full collection of blocks launched for one kernel.
- **SM**: hardware unit that schedules warps, manages registers and shared memory, and executes arithmetic instructions.
- **CUDA stream**: an ordered queue of GPU work. Work in the same stream executes in issue order, while different streams may overlap if hardware resources allow.

A timeline view in Nsight Systems or the PyTorch profiler is not local to one SM. It shows kernel launches and execution intervals across the GPU. The CUDA hardware scheduler decides how blocks from active kernels occupy SMs.

Host and Device

In this lecture, **host** means CPU-side execution, mostly Python and PyTorch runtime dispatch. **Device** means GPU-side execution.

The host launches work asynchronously. A well-behaved GPU workload should often have the host running ahead of the device: the CPU queues kernels, and the GPU consumes that queue. If the GPU finishes a kernel and has no work waiting, then a gap appears on the CUDA stream.

A simplified timeline is:

$$\text{Host dispatches kernels} \longrightarrow \text{CUDA stream queue fills} \longrightarrow \text{GPU executes kernels}$$

The dangerous pattern is:

$$\text{kernel} \longrightarrow \text{gap} \longrightarrow \text{kernel} \longrightarrow \text{gap}$$

This often means the CPU cannot feed the GPU fast enough, or that the workload contains host-device synchronization points that force the CPU and GPU to wait for each other.

Profiling Tools Mentioned

NVIDIA SMI

NVIDIA SMI utilization is coarse. Seeing 80% GPU utilization means that something is running on the GPU during 80% of sampled time. It does not prove that the GPU is using tensor cores well, saturating memory bandwidth, achieving high occupancy, or executing efficient kernels.

Nsight Systems

Nsight Systems is a system-level profiler. It shows:

- CPU threads,
- CUDA API calls,
- CUDA kernels,
- streams,
- gaps between kernels,
- rough occupancy information,
- startup and steady-state regions.

It is excellent for answering system-level questions such as:

- Is the GPU idle between kernels?
- Is the CPU launching kernels fast enough?
- Are there synchronizations?

- Are multiple streams overlapping?
- How long is startup compared with steady state?

PyTorch Profiler

The PyTorch profiler produces a Chrome trace. It has less low-level kernel detail than Nsight Systems, but it has an important advantage: it understands PyTorch source locations, operator stacks, tensor shapes, and runtime events.

This lets us map from:

$$\text{CUDA kernel} \longrightarrow \text{ATen operation} \longrightarrow \text{PyTorch module} \longrightarrow \text{Python source line}$$

That mapping is the main reason the PyTorch profiler is so useful for model optimization.

PyTorch Memory Profiler

PyTorch also has a memory profiler and snapshot visualization system. It can capture allocation lifetimes over longer windows than the full profiler because it is lighter-weight.

It helps answer:

- Which tensors are alive at peak memory?
- Are activations freed during backward?
- Are there unexpected tensors surviving longer than expected?
- Does the first step allocate more than later steps?
- Does variable sequence length cause rare memory spikes?

Main Concepts

Start with a Sanity Check, Not a Deep Optimization

The first step is simply to run the model and check whether the GPU is being used.

In the lecture, the workload reaches around 80% GPU utilization according to an NVIDIA SMI-style dashboard. This is useful, but only as a sanity check.

A high-level utilization number does not answer:

- Are the kernels large enough?
- Are there gaps between kernels?

- Are GEMMs compute-bound or memory-bound?
- Are pointwise kernels dominated by launch overhead?
- Is the host blocking on the device?
- Is the memory allocator holding tensors too long?

The correct interpretation is:

> NVIDIA SMI tells us that the GPU is active. It does not tell us whether the GPU is being used well.

Startup Versus Steady State

The first iteration is commonly much longer than later iterations. Reasons include:

- Python imports and lazy initialization,
- CUDA context creation,
- memory pool initialization,
- kernel caching,
- JIT compilation,
- autotuning,
- data loader warmup,
- cache effects.

Therefore, profiling should focus on steady state. If we optimize based on the first iteration, we may optimize one-time initialization instead of the repeated training step.

The steady-state training loop has a more regular structure:

$$\text{forward pass} \longrightarrow \text{loss computation} \longrightarrow \text{backward pass} \longrightarrow \text{optimizer step}$$

The first iteration often contains extra noise:

$$T_{\text{first}} = T_{\text{steady}} + T_{\text{init}} + T_{\text{compile}} + T_{\text{cache}} + T_{\text{allocator}}$$

where T_{first} is the first-step time and T_{steady} is the representative repeated-step time.

Using NVTX Markers

NVTX markers help connect timeline regions to code regions. Without markers, a trace is often just a sequence of kernels with names such as CUTLASS GEMM kernels, FlashAttention backward kernels, and pointwise kernels.

With NVTX ranges, one can label high-level phases:

- model forward,
- RMSNorm,
- attention,
- rotary embeddings,
- MLP,
- cross entropy,
- backward.

This turns an opaque trace into a navigable performance map.

Chrome Trace and Icicle Plot Interpretation

The PyTorch profiler trace is read as follows:

- The x-axis is time.
- Stacked bars show nested call stacks.
- CUDA kernels appear on CUDA stream rows.
- Python forward execution appears as deep Python stacks.
- Backward often appears under C++ autograd engine activity.

A useful visual heuristic is:

$$\text{deep Python source stack} \approx \text{forward pass}$$

and

$$\text{blocked Python thread with C++ autograd work} \approx \text{backward pass}$$

This distinction matters because forward and backward have different bottlenecks. Forward may contain Python overhead, dynamic shape logic, or explicit PyTorch operations. Backward may be dominated by generated autograd kernels, GEMM backward kernels, or communication if distributed training is involved.

Kernel Gaps as Evidence

A major theme is that gaps in the CUDA stream are not random. They are evidence.

If the stream contains:

$$K_1 \quad \text{gap} \quad K_2$$

then the GPU finished K_1 and had no ready work before K_2. Possible causes include:

- CPU dispatch overhead,
- Python overhead,
- PyTorch runtime overhead,
- host-device synchronization,
- data-dependent control flow,
- waiting for memory copies,
- dynamic compilation,
- data loader stalls.

The key question is:

> Is the GPU idle because the kernel work is too small, because the host is too slow, or because synchronization destroyed the host's ability to run ahead?

Hardware-Level Explanation

GPU Utilization Is Not the Same as Saturation

A GPU may report high utilization while still underperforming. For example, a workload may launch many tiny pointwise kernels. The GPU is technically active, but each kernel may be too small to occupy all SMs or amortize launch overhead.

Important distinctions:

- **Active**: some kernel is running.
- **Occupied**: many SMs have active warps.
- **Saturated**: the limiting hardware resource is being driven near its peak.
- **Efficient**: the model-level computation is using hardware in a useful way.

The hierarchy is:

$$\text{active} \not\Rightarrow \text{occupied} \not\Rightarrow \text{saturated} \not\Rightarrow \text{efficient}$$

CUDA Streams and Host Dispatch

In a good asynchronous execution pattern, the CPU queues enough kernels that the GPU always has work available. The host dispatch can run ahead of the device.

Let:

$$T_{\text{launch}}$$

be the CPU-side time to prepare and launch a kernel, and let:

$$T_{\text{kernel}}$$

be the GPU-side execution time. If T_{kernel} is large, launch overhead is hidden. If T_{kernel} is tiny, launch overhead becomes visible.

For many tiny kernels:

$$T_{\text{step}} \approx \sum_i T_{\text{launch},i} + \sum_i T_{\text{kernel},i} + \sum_j T_{\text{sync},j}$$

If the launches cannot overlap well with execution, then the GPU stream develops holes.

Host-Device Synchronization

Host-device synchronization is one of the most important red flags in the lecture.

A synchronization forces the CPU to wait until relevant GPU work completes. This destroys asynchronous slack.

Before synchronization:

$$\text{CPU queues future work while GPU executes current work}$$

After synchronization:

$$\text{CPU waits} \implies \text{GPU queue drains} \implies \text{future gaps become likely}$$

Common sources of synchronization include:

- `tensor.item()`,
- converting CUDA tensors to Python scalars,
- Python `max` or `min` involving tensors in certain ways,
- printing CUDA tensor values,
- CPU-side conditionals depending on GPU tensor values,
- explicit `torch.cuda.synchronize()`,
- some logging operations.

The lecture shows that synchronizations can appear unintentionally from innocent-looking Python code.

Memory Hierarchy

The GPU memory hierarchy relevant here is:

HBM/global memory $\rightarrow$ L2 cache $\rightarrow$ L1/shared memory $\rightarrow$ registers $\rightarrow$ compute units

Large model weights live in HBM. When batch size is small, each GEMM may load a large weight matrix from memory but perform only a small amount of computation per loaded byte. This creates memory-bandwidth-bound GEMMs.

Why Small Batch Size Hurts

For a linear layer:

$$Y = XW$$

where:

$$X \in \mathbb{R}^{M \times K}, \quad W \in \mathbb{R}^{K \times N}, \quad Y \in \mathbb{R}^{M \times N}.$$

The cost is approximately:

$$\text{FLOPs} = 2MKN.$$

The memory traffic includes loading X, loading W, and writing Y:

$$\text{Bytes} \approx b(MK + KN + MN),$$

where b is bytes per element.

When M is small, the weight matrix term KN dominates:

$$MK + KN + MN \approx KN.$$

Then arithmetic intensity becomes:

$$I = \frac{2MKN}{b(MK + KN + MN)} \approx \frac{2MKN}{bKN} = \frac{2M}{b}.$$

This means that when batch or token dimension M is small, arithmetic intensity is small. The GEMM may be memory-bandwidth-bound even though it is still a GEMM.

Mathematical Reasoning

Back-of-the-Envelope GEMM Compute Time

The lecture analyzes a GEMM with approximate shape:

$$M = 61, \quad K = 2048, \quad N = 16384.$$

The FLOP count for matrix multiplication is:

$$\text{FLOPs} = 2MKN.$$

Substituting:

$$\text{FLOPs} = 2 \cdot 61 \cdot 2048 \cdot 16384.$$

Compute this approximately:

$$2048 \cdot 16384 = 33{,}554{,}432,$$

$$61 \cdot 33{,}554{,}432 = 2{,}046{,}820{,}352,$$

$$2 \cdot 2{,}046{,}820{,}352 = 4{,}093{,}640{,}704.$$

So the GEMM requires roughly:

$$4.09 \times 10^9$$

floating point operations.

If the GPU has a theoretical peak of approximately:

$$P_{\text{peak}} = 121 \times 10^{12} \quad \text{FLOPs/s},$$

then ideal compute-bound time is:

$$T_{\text{compute}} = \frac{4.09 \times 10^{9}}{121 \times 10^{12}} \approx 3.38 \times 10^{-5} \text{ s}.$$

Converting to microseconds:

$$T_{\text{compute}} \approx 33.8 \ \mu\text{s}.$$

But the observed kernel time is closer to:

$$T_{\text{observed}} \approx 300 \ \mu\text{s}.$$

This is around:

$$\frac{300}{33.8} \approx 8.9$$

times slower than ideal compute-bound performance. This suggests that the GEMM is not primarily compute-bound.

Back-of-the-Envelope GEMM Memory Time

For BF16, each element uses:

$$b = 2$$

bytes.

A rough memory traffic estimate is:

$$\text{Bytes} = 2(MK + KN + MN).$$

For this shape:

$$MK = 61 \cdot 2048 = 124{,}928,$$

$$KN = 2048 \cdot 16384 = 33{,}554{,}432,$$

$$MN = 61 \cdot 16384 = 999{,}424.$$

Thus:

$$MK + KN + MN = 124{,}928 + 33{,}554{,}432 + 999{,}424 = 34{,}678{,}784.$$

Bytes moved:

$$\text{Bytes} = 2 \cdot 34{,}678{,}784 = 69{,}357{,}568 \approx 69.4 \text{ MB}.$$

If the L4 GPU memory bandwidth is roughly:

$$B = 300 \times 10^9 \quad \text{bytes/s},$$

then ideal memory time is:

$$T_{\text{memory}} = \frac{69.4 \times 10^6}{300 \times 10^9} \approx 2.31 \times 10^{-4} \text{ s}.$$

In microseconds:

$$T_{\text{memory}} \approx 231 \ \mu\text{s}.$$

The observed time of roughly 300 μs is much closer to the memory-bandwidth estimate than to the compute estimate. Therefore this GEMM is memory-bound.

Arithmetic Intensity

Arithmetic intensity is:

$$I = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

For the GEMM above:

$$I = \frac{2MKN}{2(MK + KN + MN)} = \frac{MKN}{MK + KN + MN}.$$

Substituting the values:

$$I = \frac{61 \cdot 2048 \cdot 16384}{61 \cdot 2048 + 2048 \cdot 16384 + 61 \cdot 16384}.$$

Using the earlier values:

$$I = \frac{2{,}046{,}820{,}352}{34{,}678{,}784} \approx 59.0 \quad \text{FLOPs/byte if counting BF16 bytes after cancellation convention.}$$

If using the explicit FLOPs and bytes definition:

$$I = \frac{4{,}093{,}640{,}704}{69{,}357{,}568} \approx 59.0 \quad \text{FLOPs/byte.}$$

The roofline threshold is:

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B}.$$

With:

$$P_{\text{peak}} = 121 \times 10^{12} \quad \text{FLOPs/s}$$

and

$$B = 300 \times 10^{9} \quad \text{bytes/s,}$$

we get:

$$I_{\text{ridge}} = \frac{121 \times 10^{12}}{300 \times 10^{9}} \approx 403 \quad \text{FLOPs/byte.}$$

Since:

$$59 \ll 403,$$

the GEMM is predicted to be memory-bound.

Roofline Model

The roofline model says:

$$P_{\text{achievable}} = \min(P_{\text{peak}}, B \cdot I),$$

where:

- $P_{\text{achievable}}$ is achievable performance,
- P_{peak} is peak compute throughput,
- B is memory bandwidth,
- I is arithmetic intensity.

If:

$$B \cdot I < P_{\text{peak}},$$

then the operation is memory-bound.

If:

$$B \cdot I \geq P_{\text{peak}},$$

then the operation is compute-bound.

In this lecture, the key insight is that a GEMM can still fall on the bandwidth-bound side of the roofline when the batch dimension is too small.

Why Increasing Batch Size Helps

Increasing batch size increases M. The FLOP count grows as:

$$\text{FLOPs} = 2MKN.$$

The weight traffic KN does not grow with M. Thus, increasing M increases reuse of the same weight matrix.

Arithmetic intensity is:

$$I(M) = \frac{2MKN}{b(MK + KN + MN)}.$$

For small M:

$$I(M) \approx \frac{2M}{b}.$$

So arithmetic intensity grows approximately linearly with M. Larger batch size gives the GPU more work per byte of weight loaded.

Code Walkthrough

Profiling with the PyTorch Profiler

A representative profiler structure is:

```
import torch
from torch.profiler import profile, ProfilerActivity

with profile(
    activities=[
        ProfilerActivity.CPU,
        ProfilerActivity.CUDA,
    ],
    record_shapes=True,
    with_stack=True,
    profile_memory=True,
) as prof:
    for step, batch in enumerate(dataloader):
        loss = training_step(model, batch)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)

        prof.step()

        if step >= 2:
            break

prof.export_chrome_trace("trace.json")
```

The key settings are:

- `ProfilerActivity.CPU`: records CPU-side events.
- `ProfilerActivity.CUDA`: records CUDA kernel events.
- `record_shapes=True`: captures tensor shapes.
- `with_stack=True`: captures source stacks.
- `profile_memory=True`: captures memory events during the profile.
- `prof.step()`: tells the profiler that a training step has completed.

The lecture emphasizes that profiles should capture only a few steps because profiler data volume is enormous. Capturing too much can make the trace

unusable.

Why Only a Few Steps Are Profiled

Profilers collect large amounts of information:

$$\text{trace size} \propto \text{number of events} \times \text{metadata per event}.$$

Events include:

- Python calls,
- ATen operations,
- CUDA launches,
- CUDA kernels,
- memory allocations,
- synchronization events,
- stack traces,
- tensor shapes.

For deep learning workloads with many small operators, this quickly becomes massive. Therefore, the practical workflow is:

1. Run long enough to reach steady state.
2. Capture a small number of representative steps.
3. Use the trace to form hypotheses.
4. Run targeted microbenchmarks or smaller profiles.

Representative Nsight Systems Command

A typical Nsight Systems command for profiling a Python training script is:

```
nsys profile \
    --trace=cuda,nvtx,osrt \
    --capture-range=cudaProfilerApi \
    --capture-range-end=stop \
    --output=profile_report \
    python train.py
```

This command records CUDA events, NVTX ranges, and OS runtime events. The exact flags may vary, but the conceptual purpose is to capture a timeline connecting CPU launch activity with GPU execution.

Using NVTX Ranges in Python

Representative instrumentation:

```
import torch

def training_step(model, batch):
    with torch.cuda.nvtx.range("forward"):
        outputs = model(batch["input_ids"])

    with torch.cuda.nvtx.range("loss"):
        loss = compute_loss(outputs, batch["labels"])

    with torch.cuda.nvtx.range("backward"):
        loss.backward()

    return loss
```

This creates readable regions in Nsight Systems and other NVTX-aware tools. The major benefit is that one can visually connect gaps and kernel clusters to the model code.

Identifying Pointwise Kernel Clusters

The lecture observes clusters of pointwise kernels, such as elementwise operations and copies. A typical unfused PyTorch expression may generate multiple kernels:

```
y = torch.cat([x1, x2], dim=-1)
z = y * scale
out = z.to(dtype=torch.bfloat16)
```

This may produce:

- a concatenation or copy kernel,
- a multiply kernel,
- a dtype conversion kernel,
- possibly additional indexing kernels.

These kernels may each be short. If each kernel takes only a few microseconds on the GPU, host launch overhead and Python overhead can dominate.

Microbenchmarking a Fused Kernel

The lecture discusses a rotary embedding-like operation called `apply_rope`. It looks fusible because it consists of copies, concatenation, arithmetic, and conversion.

A simplified representative structure is:

```
def apply_rope(x, cos, sin):
    x1 = x[..., ::2]
    x2 = x[..., 1::2]

    rotated = torch.stack((-x2, x1), dim=-1).flatten(-2)
    return x * cos + rotated * sin
```

This can create multiple pointwise and indexing kernels. A custom fused kernel may reduce GPU-side execution to a single efficient kernel.

However, the lecture shows an important trap: a more clever fused kernel can be slower end-to-end if it requires extra host-side work.

A simplified benchmark pattern is:

```
import torch
import time

def benchmark(fn, *args, warmup=10, iters=100):
    for _ in range(warmup):
        fn(*args)

    torch.cuda.synchronize()
    start = time.perf_counter()

    for _ in range(iters):
        fn(*args)

    torch.cuda.synchronize()
    end = time.perf_counter()

    return (end - start) / iters
```

The key point is:

$$T_{\text{end-to-end}} = T_{\text{host setup}} + T_{\text{launch}} + T_{\text{device execution}}.$$

A custom kernel may reduce $T_{\text{device execution}}$, but if it increases $T_{\text{host setup}}$, the total time may increase.

Why a Faster Device Kernel Can Lose

Suppose baseline PyTorch takes:

$$T_{\text{baseline}} = 50\ \mu\text{s}.$$

A custom fused kernel has a device execution time:

$$T_{\text{device}} = 2\ \mu\text{s}.$$

But if it has host-side binding, cache-key construction, dynamic compilation checks, argument preparation, or wrapper overhead, then:

$$T_{\text{custom}} = T_{\text{host setup}} + T_{\text{launch}} + 2\ \mu\text{s}.$$

If:

$$T_{\text{host setup}} + T_{\text{launch}} > 48\ \mu\text{s},$$

then the custom kernel is slower than baseline.

The lecture's conclusion is:

> Do not optimize the device kernel in isolation when the actual bottleneck is host overhead or insufficient batch size.

Memory Snapshot Collection

A representative memory snapshot workflow is:

```
import torch

torch.cuda.memory._record_memory_history(enabled=True)

for step, batch in enumerate(dataloader):
    loss = training_step(model, batch)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)

    if step >= 5:
        break

torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")
torch.cuda.memory._record_memory_history(enabled=False)
```

The memory profiler can be used for longer spans because it is lighter than the full profiler. The output can be loaded into PyTorch's memory visualization tool.

Understanding the Memory Timeline

In the memory timeline:

- model weights appear as a static base allocation,

- activations grow during the forward pass,
- activations are released during the backward pass,
- gradients and optimizer states may appear depending on training type,
- rare long examples can create spikes.

For LoRA fine-tuning, gradients are small because only low-rank adapter parameters are trained. Therefore, activation memory dominates more than full-model gradient memory.

Deleting Logits to Shorten Tensor Lifetime

The lecture identifies a case where a large logits tensor remains alive longer than expected because a Python variable still references it.

A simplified problematic pattern is:

```
def training_step(model, batch):
    logits = model(batch["input_ids"])
    loss = chunked_cross_entropy(logits, batch["labels"])
    return loss
```

Even after the loss is computed, `logits` remains a live Python variable until its scope ends or it is overwritten. PyTorch cannot free the underlying GPU memory while the Python object still holds a reference.

A simple fix is:

```
def training_step(model, batch):
    logits = model(batch["input_ids"])
    loss = chunked_cross_entropy(logits, batch["labels"])

    del logits

    return loss
```

This is a promise that the code will not use `logits` later. It decrements the Python reference count. If this was the last reference, the tensor object can be freed and the CUDA memory can be returned to the allocator.

Why Python Reference Counts Matter

A tensor's GPU memory cannot be freed while a Python object still references it. The Python object acts as a handle to the underlying allocation.

A simplified model is:

$$\text{GPU allocation is live} \iff \text{reference count} > 0.$$

If a variable remains in scope, the reference count remains positive. Therefore:

`del logits` $\implies$ decrement reference count $\implies$ possibly release tensor memory.

The compiler cannot always prove that a tensor is dead because Python allows dynamic behavior such as closures, attributes, mutation, and reflection.

For example:

```
def f():
    logits = model(x)

    def g():
        return logits

    loss = criterion(logits, y)
    return loss, g
```

Here `logits` is captured by a closure. It cannot be safely freed after loss computation because `g` may later access it.

Another example:

```
self.cached_logits = logits
```

Now the tensor escapes the local function scope. A compiler cannot generally assume it is dead.

Avoiding Python Scalar Synchronization

The lecture highlights a subtle issue involving Python `max`. Consider:

```
denom = max(1, x)
```

If `x` is a CUDA tensor or depends on CUDA tensor values, Python may need a concrete host value. This can cause a host-device synchronization.

A safer tensorized form is:

```
denom = torch.maximum(
    torch.ones_like(x),
    x,
)
```

or, if the scalar shape is appropriate:

```
denom = torch.maximum(
    torch.ones((), device=x.device, dtype=x.dtype),
    x,
)
```

The point is to keep the computation on the GPU and avoid converting GPU values into Python scalars.

Avoiding `torch.tensor` on Python Lists in the Hot Path

Another issue appears in a zero-padding operation. The code uses something like:

```
padding = torch.tensor(padding_values, device=x.device)
```

If `padding_values` is a Python list, then the CPU must build the tensor and transfer it to the GPU. If this happens inside the training step, it introduces overhead and possible synchronization.

A better pattern is to initialize static tensors once:

```
class Module(torch.nn.Module):
    def __init__(self, padding_values):
        super().__init__()
        self.register_buffer(
            "padding_values",
            torch.tensor(padding_values),
            persistent=False,
        )

    def forward(self, x):
        padding = self.padding_values.to(device=x.device)
        return use_padding(x, padding)
```

If the device is known at initialization or after module transfer, one can keep the tensor on the device in steady state. The important idea is:

$$\text{static metadata} \quad \Longrightarrow \quad \text{prepare once, not every iteration.}$$

Performance Intuition

Optimization Should Follow the Bottleneck

The lecture gives a clear example of avoiding premature kernel optimization.

The trace shows pointwise kernels and gaps. A natural reaction is to write a fused kernel. But the fused kernel requires extra host machinery and is slower end-to-end. The correct bottleneck is not only the pointwise kernel execution; it is the small amount of work per launch and insufficient batching.

The lesson is:

A fused kernel is valuable when kernel launch overhead and memory traffic dominate and when the fused implementation does not add more host overhead than it removes.

Batch Size as a Systems Optimization

Increasing batch size helps in two ways:

1. It increases work per kernel, making launch overhead less important.
2. It increases arithmetic intensity for GEMMs by reusing the same weights across more activations.

For a memory-bound GEMM, larger M can move the operation closer to the compute-bound region.

If the model has batch size 1, many kernels are too small. If batch size increases to 2, throughput may improve even if each step becomes slightly slower because each step processes twice as many examples.

If:

$$\text{batch size} = 1, \quad \text{throughput} = 12 \text{ batches/s},$$

then examples per second are:

$$12 \cdot 1 = 12.$$

If:

$$\text{batch size} = 2, \quad \text{throughput} = 8 \text{ batches/s},$$

then examples per second are:

$$8 \cdot 2 = 16.$$

So even though batches per second decreased, examples per second improved.

Memory Limits from Variable Sequence Lengths

The model cannot simply increase batch size indefinitely because rare long sequences create memory spikes.

Transformer activation memory often scales with sequence length S, hidden dimension H, number of layers L, and batch size B. A simplified activation scaling model is:

$$M_{\text{act}} \propto B \cdot S \cdot H \cdot L.$$

Attention may have additional sequence-dependent costs depending on implementation:

$$M_{\text{attention}} \propto B \cdot A \cdot S^2$$

for standard attention materializing attention matrices, where A is number of heads. FlashAttention reduces this memory pressure, but sequence length still affects activations and compute.

If most samples have:

$$S \in [100, 200],$$

but rare samples have:

$$S \approx 1000,$$

then peak memory is dominated by rare long samples. This can prevent using a larger batch size even though most steps would fit.

Dynamic Local Batch Size

The lecture suggests an optimization: use a larger normal batch size, but when a rare long sequence appears, split it into smaller local sub-batches and use gradient accumulation.

Suppose global effective batch size is:

$$B_{\text{eff}} = B_{\text{local}} \cdot A,$$

where A is the number of gradient accumulation steps.

If a batch contains long sequences, reduce B_{local} temporarily and increase accumulation so that:

$$B_{\text{eff}}$$

remains approximately stable.

This is especially attractive for LoRA because trainable gradients are tiny compared with full fine-tuning. Gradient accumulation is less memory-expensive in LoRA than in full-model training.

Gradient Accumulation Trade-Off

Normally, gradient accumulation can increase memory pressure because gradients must persist across microsteps. In full fine-tuning, gradients for all model parameters are large.

For full training:

$$M_{\text{grad}} \approx M_{\text{weights}}.$$

For Adam-style optimizers, optimizer state can be even larger:

$$M_{\text{optimizer}} \approx 2M_{\text{weights}}$$

for first and second moments in FP32.

But in LoRA fine-tuning, only low-rank adapter parameters receive gradients. If LoRA rank is r, and the adapted layer has dimensions d_{in} and d_{out}, then LoRA trainable parameters are:

$$rd_{\text{in}} + rd_{\text{out}} = r(d_{\text{in}} + d_{\text{out}}).$$

The original dense weight has:

$$d_{\text{in}}d_{\text{out}}$$

parameters. Since:

$$r(d_{\text{in}} + d_{\text{out}}) \ll d_{\text{in}}d_{\text{out}},$$

LoRA gradients are small. Therefore, gradient accumulation is more acceptable.

Gemma MLP Fusion Opportunity

The lecture notes that Gemma's MLP structure has two linear projections with the same shape. This creates an opportunity to concatenate weights and perform one larger matrix multiplication, then split the result.

A common gated MLP pattern is:

$$\text{MLP}(x) = W_{\text{down}} \left(\sigma(xW_{\text{gate}}) \odot xW_{\text{up}} \right).$$

The two projections:

$$xW_{\text{gate}} \quad \text{and} \quad xW_{\text{up}}$$

may have the same shape. Instead of launching two GEMMs:

$$G = xW_{\text{gate}}, \quad U = xW_{\text{up}},$$

we can concatenate:

$$W_{\text{cat}} = \begin{bmatrix} W_{\text{gate}} & W_{\text{up}} \end{bmatrix}.$$

Then compute:

$$Z = xW_{\text{cat}}.$$

Finally split:

$$Z = \begin{bmatrix} G & U \end{bmatrix}.$$

This may be faster because one larger GEMM can have better arithmetic intensity and lower relative launch overhead than two smaller GEMMs.

If one individual GEMM takes time T, two separate GEMMs cost approximately:

$$T_{\text{separate}} \approx 2T.$$

If the fused GEMM costs:

$$T_{\text{fused}} \approx 1.5T,$$

then the speedup is:

$$\text{speedup} = \frac{2T}{1.5T} = \frac{4}{3} \approx 1.33.$$

This is a model-specific optimization. It is more semantics-preserving than quantization because it computes the same mathematical projections, only scheduled differently.

Common Misconceptions

Misconception: High GPU Utilization Means Good Performance

High NVIDIA SMI utilization only means the GPU is doing something. It does not prove that the model is using the GPU efficiently.

A model can have high utilization but still suffer from:

- small kernels,
- stream gaps,
- synchronization,
- memory-bound GEMMs,
- poor batching,
- excessive host overhead.

Misconception: GEMM Means Compute-Bound

GEMM is often compute-heavy, but small-batch GEMM can be memory-bound.

The determining quantity is arithmetic intensity:

$$I = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

If I is below the roofline ridge point, the GEMM is memory-bound.

Misconception: Fusion Always Helps

Fusion helps when it reduces memory traffic, launch overhead, or intermediate materialization without adding excessive host overhead.

Fusion may hurt when:

- the operation is too small,
- the fused kernel requires expensive Python-side setup,
- dynamic dispatch or cache-key logic dominates,
- the baseline is already hidden behind larger GPU work,
- the actual bottleneck is batch size or synchronization.

Misconception: The Compiler Will Fix Everything

Compilers can fuse operations, remove dead tensors, and reduce overhead in some cases. But Python is dynamic. Tensors can escape through closures, object attributes, global variables, mutation, or reflection.

Therefore, one should not rely entirely on `torch.compile` or any compiler to prove all tensor lifetimes or remove all host overhead.

Misconception: CPU Utilization Should Be High

For model execution, high CPU utilization is usually not the goal. The CPU is often just dispatching GPU work from a single thread.

High CPU utilization may be normal if data loaders are doing parallel preprocessing. But model-side execution should not require heavy CPU work. The goal is not to maximize CPU utilization; the goal is to keep the GPU fed.

Misconception: The First Step Represents Training Performance

The first step often includes initialization, caching, allocation, and compilation. Steady-state performance should be measured after warmup.

Practical Takeaways

Profiling Workflow

A practical profiling workflow is:

1. Run the model normally and sanity-check GPU usage.
2. Use Nsight Systems to inspect timeline structure, startup, steady state, gaps, streams, and synchronizations.
3. Add NVTX markers to make high-level regions visible.
4. Use the PyTorch profiler to map kernels back to PyTorch operators and source code.
5. Focus on steady-state steps, not the first iteration.
6. Look for CUDA stream gaps.
7. Search traces for synchronization events.
8. Inspect tensor shapes for hot operations.
9. Use datasheet math to classify kernels as compute-bound or memory-bound.
10. Use the memory profiler to understand peak memory and tensor lifetimes.

11. Validate suspected optimizations with microbenchmarks.
12. Prefer semantics-preserving optimizations before destructive ones such as quantization.

What to Look for in a Trace

Important patterns include:

- long startup region,
- regular steady-state iteration pattern,
- large GEMMs,
- pointwise kernel clusters,
- gaps between kernels,
- host-device synchronization,
- memory allocation spikes,
- first-step anomalies,
- variable sequence length effects,
- tensors surviving past expected lifetime.

When to Consider Batch Size

Consider increasing batch size when:

- GEMMs are memory-bound due to small M,
- pointwise kernels are too small,
- host launch overhead is visible,
- GPU stream has gaps,
- activation memory has headroom for typical examples.

But check:

- rare long sequence lengths,
- peak memory,
- activation scaling,
- gradient accumulation cost,
- data loader behavior.

When to Consider Fusion

Consider fusion when:

- multiple pointwise kernels operate on the same tensors,
- intermediate tensors are large,
- memory traffic dominates,
- launch overhead is significant,
- the fused kernel has low host overhead,
- the operation is repeated many times.

Avoid jumping to fusion when:

- the fused kernel needs expensive setup,
- the operation is too small,
- batching would solve the issue more directly,
- synchronization is the true bottleneck.

When to Consider Quantization

Quantization can help memory-bound workloads because smaller dtypes reduce memory traffic.

For example, changing BF16 weights to INT8 changes bytes per element from:

$$2 \quad \text{bytes}$$

to:

$$1 \quad \text{byte.}$$

INT4 changes it to:

$$0.5 \quad \text{bytes.}$$

For a memory-bound layer, reducing bytes can reduce time approximately proportionally, assuming dequantization and packing overheads do not dominate.

However, quantization changes numerical semantics. The lecture's philosophy is to first try semantics-preserving optimizations, such as batching, removing synchronizations, fixing tensor lifetimes, and fusing mathematically equivalent operations. Then consider quantization after understanding accuracy trade-offs.

Project or Experiment Ideas

Experiment 1: Profile a Small PyTorch Model

Take a small transformer or MLP and profile it with the PyTorch profiler. Record shapes and stacks. Identify:

- forward region,
- backward region,
- largest GEMMs,
- pointwise clusters,
- CUDA stream gaps.

Experiment 2: Batch Size and Arithmetic Intensity

Benchmark a linear layer:

$$Y = XW$$

for different values of M, while keeping K and N fixed. Measure runtime and compute:

$$I(M) = \frac{2MKN}{b(MK + KN + MN)}.$$

Observe how larger M improves throughput.

Experiment 3: Host Synchronization from `.item()`

Compare:

```
if loss.item() < threshold:
    do_something()
```

against a version that avoids per-step scalar extraction. Profile both and search for synchronization events.

Experiment 4: Python `max` Versus Tensorized `torch.maximum`

Create a CUDA tensor and compare Python scalar-style logic against tensorized logic. Inspect whether host-device synchronization appears.

Experiment 5: Tensor Lifetime and `del`

Construct a function that creates a large tensor, computes a loss, and then either deletes or does not delete the intermediate. Use the PyTorch memory profiler to inspect allocation lifetimes.

Experiment 6: Fusing Two Linear Projections

Implement two separate linear layers:

$$G = XW_1, \quad U = XW_2.$$

Then implement one concatenated linear layer:

$$Z = X[W_1 \; W_2],$$

followed by splitting. Benchmark both for different batch sizes.

Experiment 7: Variable Sequence Length Memory Spikes

Create batches with mostly short sequences and rare long sequences. Use the memory profiler to show how rare long examples dominate peak memory.

Final Mental Model

The lecture's final mental model is:

> Profiling is the process of converting a vague feeling that "the GPU is slow" into a precise causal story. You start from the timeline, identify where time and memory go, connect kernels back to source code, classify bottlenecks with hardware math, and only then choose an optimization.

The key causal chain in this lecture is:

small batch size $\implies$ small kernels and low GEMM arithmetic intensity $\implies$ host overhead and memo

plus:

host-device synchronization $\implies$ loss of asynchronous slack $\implies$ CUDA stream gaps

plus:

Python references to tensors $\Longrightarrow$ tensor memory remains live $\Longrightarrow$ unexpected memory pressure

A strong GPU performance engineer does not merely ask, "Which kernel is slow?" A strong engineer asks:

- Is the workload in steady state?
- Is the GPU waiting on the CPU?
- Is the CPU waiting on the GPU?
- Are kernels large enough?
- Are GEMMs compute-bound or memory-bound?
- Are tensor shapes causing poor reuse?
- Are rare examples setting the memory limit?
- Are Python variables keeping tensors alive?
- Are we optimizing the bottleneck or just the most visible kernel?

The practical conclusion is:

> Use profilers to build a story. Use math to test the story. Use code changes to validate the story. Only then optimize aggressively.

Visual Concept

Draw a three-level profiling stack. At the top, show Python model code and PyTorch modules. In the middle, show PyTorch profiler mapping operations to source lines. At the bottom, show CUDA streams with GEMM kernels, pointwise kernels, synchronization markers, and empty gaps. Add arrows from gaps back to Python overhead and synchronization.

Visual Concept

Draw a roofline plot with arithmetic intensity on the x-axis and throughput on the y-axis. Mark a small-batch GEMM on the bandwidth-bound slope. Then draw an arrow to the right showing that increasing batch size increases arithmetic intensity and moves the GEMM closer to the compute roof.

Visual Concept

Draw a memory timeline. Show weights as a static block at the bottom. Show activations rising during forward and falling during backward. Then show an anomalous large logits block that remains alive across the step until `del logits` removes the Python reference.